

Topic 6: LangGraph & State Contracts — Parallel Reducers

TOPIC OBJECTIVE

Multi-agent coordination requires strict state contracts and race-free concurrency. Using LangGraph (src/graph.py), AegisClinical compiles a Directed Acyclic Graph (DAG) with fan-out parallel edges and a fan-in barrier synchronization. By defining state with typing.Annotated and operator.add (src/schemas.py), concurrent nodes append findings without write contention.

1. The Real-World Analogy: The Trauma Bay Whiteboard

Imagine an Emergency Department Trauma Bay. When a critical patient arrives, three specialist physicians rush to the bedside at the exact same moment: the **Lab Doctor**, the **Pharmacist**, and the **Medical Historian**.

The Danger of a Single Pen (Race Condition):

If there is only one pen and one blank space on the whiteboard labeled 'Notes', each doctor will erase what the previous doctor just wrote. The Pharmacist will overwrite the critical blood test numbers.

The AegisClinical Solution (The Reducer Pattern):

Instead of a shared blank space, the Charge Nurse sets up three separate bulletin boards with sticky-note slots:

- The Lab Doctor pins her KDIGO AKI alert to the Lab Board.
- The Pharmacist pins his Triple Whammy alert to the Pharmacy Board.
- The Historian pins active hypertension to the History Board.

The Synchronization Barrier: The Head Trauma Surgeon (triage_coordinator) stands in the center and does not make the final diagnosis until all three specialist boards have their notes posted.

2. Compiling the Graph: src/graph.py

LangGraph represents execution as a state machine with nodes and edges. Here is the entire graph compilation logic:

```
def create_clinical_graph():
    builder = StateGraph(ClinicalGraphState)

    # 1. Register specialized agent nodes
    builder.add_node('lab_agent', lab_agent_node)
    builder.add_node('pharma_agent', pharma_agent_node)
    builder.add_node('history_agent', history_agent_node)
    builder.add_node('triage_coordinator', triage_coordinator_node)

    # 2. Scatter: Fan-out from START to 3 parallel agents
    builder.add_edge(START, 'lab_agent')
    builder.add_edge(START, 'pharma_agent')
    builder.add_edge(START, 'history_agent')

    # 3. Gather: Fan-in barrier to Lead Triage Coordinator
    builder.add_edge('lab_agent', 'triage_coordinator')
    builder.add_edge('pharma_agent', 'triage_coordinator')
    builder.add_edge('history_agent', 'triage_coordinator')

    builder.add_edge('triage_coordinator', END)
    return builder.compile()
```

3. The Pydantic Data Contracts: src/schemas.py

Every item emitted by an agent is strictly typed via **Pydantic v2**. This prevents schema drift:

Model	Emitted By	Key Schema Fields
LabAlert	lab_agent	name, current_value, baseline_value, alert_type, severity
DrugInteraction	pharma_agent	drugs, severity, mechanism, recommendation
ChronicCondition	history_agent	code, display, status
TriageAssessment	triage_coordinator	triage_level, summary, action_items

KNOWLEDGE CHECK & NEXT STEP

- 1. What does `builder.add_edge('lab_agent', 'triage_coordinator')` do?**
It specifies that triage_coordinator must wait for lab_agent to complete before executing.
- 2. What is the role of Pydantic models vs. raw dictionaries?**
Pydantic validates types at runtime, preventing malformed data (like string lab values or missing severity ratings) from entering clinical state.
- 3. Next Step in Topic 7: Local SLM Inference — How we bridge to local Ollama with zero external dependencies and graceful fallbacks.**